

System Module Breakdown and Simplification Analysis

This report analyses the current **HUEY_P-EAFIX** trading system based on the provided process contract and project sources. It identifies each module and sub-module, explains their responsibilities, key functions, dependencies and relationships, and assesses opportunities to simplify the design while preserving minimum viable trading functionality.

Note: References like **【turn2file0†L1-L17】** refer to passages in the official design documents or code files that describe the system's components. These citations are retained for traceability.

Module structure and responsibilities

1. F1_CONFIG_PREFERENCES (configuration service)

- **Purpose:** Resolve a stable configuration snapshot from default values and overrides.
- **Responsibilities:**
 - Combine the base configuration and overrides (environment variables, CLI flags).
 - Validate against a schema and ensure deterministic ordering.
 - Produce an immutable `ResolvedConfig` and compute a hash for versioning **【turn2file0†L1-L17】** .
- **Key functions/services:**
 - `configuration_service.py` exposes functions to load defaults, apply overrides and validate the resulting configuration.
- **Dependencies:**
 - Consumes configuration sources (e.g., YAML files, environment).
 - Publishes `ResolvedConfig` consumed by every downstream module.

2. F3_CLOCK_SCHEDULER (scheduler)

- **Purpose:** Generate `ScheduleTick` events at configured cadences.
- **Responsibilities:**
 - Use the resolved configuration to determine polling cadence (e.g., every minute).
 - Support frozen-time mode for deterministic tests **【turn2file0†L1-L17】** .
 - Emit health alerts on stall conditions.
- **Key functions/services:**
 - `scheduler.py` orchestrates time-based ticks and logs health.
- **Dependencies:**
 - Consumes `ResolvedConfig`.
 - Feeds `ScheduleTick` into calendar polling (D2) and anticipator (D4).

3. D2_CALENDAR_SOURCE_ADAPTER (calendar source adapter)

- **Purpose:** Poll external economic calendars and convert them to raw events.
- **Responsibilities:**
 - Make HTTP calls to configured calendar providers and parse responses.
 - Ensure the response is a 200 OK with parseable payload; emit `IngestError` on failure `【turn2file0†L1-L17】`.
 - Forward `CalendarRaw` objects containing event IDs, times and metadata.
- **Key functions/services:**
 - `calendar_service.py::poll_scheduler` handles polling and back-off logic.
- **Dependencies:**
 - Consumes `CalendarSourceConfig` and `ScheduleTick`.
 - Provides `CalendarRaw` to the normalizer.

4. D3_CALENDAR_NORMALIZER (calendar normalizer)

- **Purpose:** Normalize raw calendar entries to canonical `CalendarEvent` objects.
- **Responsibilities:**
 - Convert timestamps to UTC and map vendor-specific fields into standardized fields such as impact, symbol and deduplication keys `【turn2file0†L1-L17】`.
 - Apply de-duplication rules.
 - Emit `NormalizationError` for malformed rows while continuing with valid events.
- **Key functions/services:**
 - `calendar_service.py::normalize_events`.
- **Dependencies:**
 - Consumes `CalendarRaw` and `ResolvedConfig`.
 - Outputs `CalendarEvent` to event logging and trigger builder.

5. F2_EVENT_LOG (event log)

- **Purpose:** Persist calendar events and other event streams in an append-only log.
- **Responsibilities:**
 - Enforce idempotency by message ID, order events by `created_utc` and a tie-breaker, and roll back on errors `【turn2file0†L1-L17】`.
 - Provide an immutable audit trail.
- **Key functions/services:**
 - `event_log.py` implements writing and recovery.
- **Dependencies:**
 - Consumes `CalendarEvent` and other domain events.
 - Exposes `EventStream` to downstream modules (trigger builder, OMS).

6. D4_CALENDAR_TRIGGER_BUILDER (calendar anticipator)

- **Purpose:** Build anticipation triggers from calendar events.
- **Responsibilities:**
 - Use `CalendarEvent`, `ScheduleTick` and configuration to compute windows around events (e.g., 30 minutes before/after) and suppression metadata.
 - Ensure there is no duplicate trigger for the same event/offset and enforce TTL rules.

- Default to conservative suppression on error `turn2file0†L1-L17` .
- **Key functions/services:**
- `calendar_anticipator.py` calculates symbol sets and windows.
- **Dependencies:**
- Consumes events from the event log and `ScheduleTick` .
- Provides `CalendarTrigger` to the feature packager.

7. D1_MARKET_FEED_ADAPTER (market feed adapter)

- **Purpose:** Ingest live market ticks.
- **Responsibilities:**
- Receive raw tick messages from a broker or price feed, normalize symbols, validate timestamps and reject malformed data `turn2file0†L1-L17` .
- Emit `MarketTick` objects.
- **Key functions/services:**
- `market_feed_adapter.py` with deterministic timestamp freshness checks.
- **Dependencies:**
- Consumes `RawTick` and `ResolvedConfig` .
- Feeds `MarketTick` to the bar builder.

8. C1_BAR_BUILDER (bar builder)

- **Purpose:** Aggregate ticks into OHLC bars.
- **Responsibilities:**
- Define deterministic boundaries (e.g., 1-minute bars) and apply tie-breaker rules.
- Drop out-of-order ticks per policy and emit error events `turn2file0†L1-L17` .
- **Key functions/services:**
- `bar_builder.py` .
- **Dependencies:**
- Consumes `MarketTick` and `ResolvedConfig` .
- Outputs `Bar` objects consumed by the indicator engine.

9. C2_INDICATOR_ENGINE (indicator engine)

- **Purpose:** Compute technical indicators.
- **Responsibilities:**
- Calculate moving averages, volatility bands, expiry indicators, etc., using deterministic rounding and parameters pinned to the configuration snapshot `turn2file0†L1-L17` .
- Emit complete `IndicatorSnapshot` objects or an error; partial snapshots are not output unless flagged.
- **Key functions/services:**
- `indicator_engine.py` and support modules (e.g., expiry indicator service).
- **Dependencies:**
- Consumes `Bar` and `ResolvedConfig` .
- Outputs `IndicatorSnapshot` to the feature packager.

10. C3_FEATURE_PACKAGER (feature packager)

- **Purpose:** Assemble the strategy feature frame.
- **Responsibilities:**
 - Combine indicator snapshots, calendar triggers and configuration into a `FeatureFrame` containing the exact declared feature set with stable ordering `【turn2file0†L1-L17】`.
 - Emit packaging errors and skip invalid frames.
- **Key functions/services:**
 - `feature_packager.py`.
- **Dependencies:**
 - Consumes `IndicatorSnapshot`, `CalendarTrigger` and `ResolvedConfig`.
 - Emits `FeatureFrame` to the signal engine.

11. S1_SIGNAL_ENGINE (signal generator)

- **Purpose:** Generate entry signals or suppressions based on the feature frame.
- **Responsibilities:**
 - Take the feature frame, current position summary and configuration to determine whether to produce a `Signal` (long or short) or a `SignalSuppressed` with enumerated reasons `【turn2file0†L1-L17】`.
 - Exclude sizing or broker-specific details.
 - Emit suppression with deterministic reason code on internal errors.
- **Key functions/services:**
 - `signal_generator.py` and `compliance/auto-remediation` functions for auto-remediation.
- **Dependencies:**
 - Consumes `FeatureFrame`, `PositionSummary` and `ResolvedConfig`.
 - Outputs to the intent builder.

12. S2_INTENT_BUILDER (intent builder)

- **Purpose:** Convert signals into trade intents.
- **Responsibilities:**
 - Build a `TradeIntent` with a stable correlation ID copied from the signal and attach constraints and rationale `【turn2file0†L1-L17】`.
 - Reject invalid intents and emit errors.
- **Key functions/services:**
 - `intent_builder.py`.
- **Dependencies:**
 - Consumes `Signal` and `ResolvedConfig`.
 - Provides `TradeIntent` to the risk evaluator.

13. R1_RISK_EVALUATOR (risk evaluator)

- **Purpose:** Evaluate risk and size trades.
- **Responsibilities:**
 - Take a trade intent, portfolio state and risk policy to approve or reject and compute sized parameters with deterministic rounding `【turn2file0†L1-L17】`.
 - Provide reasons for rejection (e.g., exposure limits).

- Default to `REJECT` with a `SAFETY_FAIL` reason if rule evaluation fails.
- **Key functions/services:**
- `risk_manager.py` implements rule engine and sizing.
- **Dependencies:**
- Consumes `TradeIntent`, `PortfolioState`, `RiskPolicy` and `ResolvedConfig`.
- Outputs `RiskDecision` to the order intent compiler.

14. R2_ORDER_INTENT_COMPILER (order intent compiler)

- **Purpose:** Compile sized trade decisions into broker-ready order intents.
- **Responsibilities:**
- Create `OrderIntent` objects with unique `client_order_id` and `idempotency_key` according to policy `turn2file0†L1-L17`.
- Emit errors and stop if compilation fails.
- **Key functions/services:**
- `order_intent_compiler.py`.
- **Dependencies:**
- Consumes `RiskDecision` and `ResolvedConfig`.
- Passes `OrderIntent` to the order router.

15. O1_ORDER_ROUTER (order router)

- **Purpose:** Select the appropriate broker or route for the order.
- **Responsibilities:**
- Apply broker policy and deterministic routing to preserve the order intent hash, enforce idempotency, and reject orders on failure `turn2file0†L1-L17`.
- **Key functions/services:**
- `order_router.py`.
- **Dependencies:**
- Consumes `OrderIntent`, `BrokerPolicy` and `ResolvedConfig`.
- Emits `RoutedOrderIntent` to the transport layer.

16. B1_MT4_ADAPTER_TRANSPORT (transport service)

- **Purpose:** Serialize and send orders to MT4 (EA) using CSV or socket transport.
- **Responsibilities:**
- Form a `BrokerOrderEnvelope` containing routing information, sequence numbers and checksums; perform atomic write+rename for CSV to avoid partial reads `turn2file0†L165-L168`.
- Handle deterministic retry schedules and emit `TransportError` on persistent failure.
- Implementation details include file names, `file_seq`, `checksum`, `hybrid_id` and socket heartbeat/demotion logic `turn3file9†L74-L90`.
- **Key functions/services:**
- `transport_service.py`.
- **Dependencies:**
- Consumes `RoutedOrderIntent` and `ResolvedConfig`.
- Sends envelopes to the EA and returns `AdapterAck` to the orchestrator.

17. B2_MT4_EA_EXECUTOR (EA executor)

- **Purpose:** Execute broker orders via the MT4 Expert Advisor (EA).
- **Responsibilities:**
 - Interpret `BrokerOrderEnvelope`, send orders to the broker and emit `BrokerExecEvent` with statuses such as accept, fill, reject or close [\[turn2file0†L97-L107\]](#).
 - Ensure no strategy or risk logic exists in the EA; it only executes.
 - Provide `EAHealth` heartbeats.
- **Key functions/services:**
 - EA code (e.g., `EAFIX.mq4` or `HUEY_P_EA_ExecutionEngine_v7.1_Enhanced.mq4`).
- **Dependencies:**
 - Consumes envelopes from the transport.
 - Emits broker events to the normalizer.

18. B3_EXEC_EVENT_NORMALIZER (execution event normalizer)

- **Purpose:** Normalize broker events into canonical execution reports.
- **Responsibilities:**
 - Deduplicate events, order them, and round fields according to canonical rules, producing `ExecutionReport` and `PositionSnapshot` [\[turn2file0†L97-L107\]](#).
 - Quarantine malformed broker events and emit `NormalizationError`.
- **Key functions/services:**
 - `result_processor.py`.
- **Dependencies:**
 - Consumes `BrokerExecEvent` and `ResolvedConfig`.
 - Provides normalized reports to the OMS state machine.

19. O2_OMS_STATE_MACHINE (OMS state machine)

- **Purpose:** Apply execution reports to order and position state.
- **Responsibilities:**
 - Update order states (e.g., new, partially filled, filled, cancelled) and positions (open size, entry price) in an idempotent manner; handle out-of-order events [\[turn2file0†L97-L107\]](#).
 - Emit `TradeClosedRaw` when positions close.
 - Quarantine streams on inconsistency.
- **Key functions/services:**
 - `oms.py`.
- **Dependencies:**
 - Consumes `RoutedOrderIntent`, `ExecutionReport`, `PositionSnapshot` and `ResolvedConfig`.
 - Outputs state changes and raw trade closes.

20. O3_TRADE_CLOSE_CLASSIFIER (trade close classifier)

- **Purpose:** Classify closed trades and compute canonical profit and loss (PnL).
- **Responsibilities:**
 - Determine the close reason (take-profit, stop-loss, manual) and apply deterministic PnL rounding [\[turn2file0†L97-L107\]](#).

- Emit computation error flags if PnL cannot be determined.
- **Key functions/services:**
- `trade_close_classifier.py`.
- **Dependencies:**
- Consumes `TradeClosedRaw` and `ResolvedConfig`.
- Produces `TradeClosed` for outcome bucketing.

21. E1_OUTCOME_BUCKETIZER (outcome bucketizer)

- **Purpose:** Bucketize trade outcomes for reinforcement learning and performance analysis.
- **Responsibilities:**
- Map `TradeClosed` to enumerated buckets (e.g., big-win, small-win, loss) with stable mapping `【turn2file0†L97-L107】`.
- Emit `UNKNOWN` bucket and quarantine records if mapping fails.
- **Key functions/services:**
- `outcome_bucketizer.py`.
- **Dependencies:**
- Consumes `TradeClosed` and `ResolvedConfig`.
- Outputs `OutcomeBucket` for matrix lookup.

22. E2_PROXIMITY_EVALUATOR (event proximity evaluator)

- **Purpose:** Compute the proximity of a trade to calendar events.
- **Responsibilities:**
- Determine the time distance between current ticks (`ClockTick`) and nearby `CalendarEvent`; classify into thresholds (e.g., near, far) and validate no overlap or gaps `【turn2file0†L97-L107】`.
- Default to the most conservative bucket on error.
- **Key functions/services:**
- `proximity_evaluator.py`.
- **Dependencies:**
- Consumes `CalendarEvent`, `ClockTick` and `ResolvedConfig`.
- Produces `EventProximity` used for matrix decisions.

23. E3_MATRIX_LOOKUP (matrix lookup)

- **Purpose:** Choose a re-entry or follow-on strategy based on outcome buckets and event proximity.
- **Responsibilities:**
- Combine `OutcomeBucket`, `EventProximity`, `SignalContext` and a matrix profile to select a `MatrixDecision`, or use a fallback if the matrix is missing `【turn2file0†L97-L107】`.
- Ensure deterministic fallback hierarchy and pinned matrix version.
- **Key functions/services:**
- `matrix_lookup.py`.
- **Dependencies:**
- Consumes `OutcomeBucket`, `EventProximity`, `SignalContext`, `MatrixProfile` and `ResolvedConfig`.
- Provides `MatrixDecision` for reentry intent.

24. E4_REENTRY_INTENT_BUILDER (reentry intent builder)

- **Purpose:** Construct reentry trade intents (or suppress them).
- **Responsibilities:**
 - Accept a `MatrixDecision`, reentry chain state and configuration to either create a new `TradeIntent` or produce `ReentrySuppressed` (with deterministic reasons) [turn2file0†L97-L107] .
 - Enforce hard caps such as no more than two re-entry levels and no circular chains.
- **Key functions/services:**
 - `reentry_intent_builder.py`.
- **Dependencies:**
 - Consumes `MatrixDecision`, `ReentryChainState` and `ResolvedConfig`.
 - Emits new trade intents fed back into the main pipeline (Step 25).

25. Loop (flow orchestrator)

- **Purpose:** Orchestrate the trading loop for both primary and reentry trades.
- **Responsibilities:**
 - Serve as the central control plane that ties together signal handling, risk evaluation, order routing, execution monitoring, reentry decisions and event scheduling [turn2file1†L213-L228] .
 - Enforce chain depth and velocity limits to prevent runaway loops.
 - Manage idempotency keys across iterations.
- **Key functions/services:**
 - The orchestrator (sometimes called `flow-orchestrator` or `orchestrator`) coordinates calls to other modules.
- **Dependencies:**
 - Consumes trade intents from the reentry builder and calls the risk→order→transport→execution chain again until the chain is complete or suppressed.

26. P1_HEALTH_AGGREGATOR (health service)

- **Purpose:** Aggregate health signals across the system and evaluate SLOs.
- **Responsibilities:**
 - Combine `AdapterHealth`, `ModuleHealth` and configuration thresholds to produce a `HealthReport` [turn2file0†L97-L107] .
 - Default to a risk-off posture (suppress new intents) if health is unknown or degraded.
- **Key functions/services:**
 - `health_service.py`.
- **Dependencies:**
 - Consumes health metrics from transport, modules and environment.
 - Produces `HealthReport` consumed by the orchestrator and telemetry.

Relationships and interactions

The modules form a pipeline that processes external data (calendar and market ticks), generates signals, sizes and routes orders, transmits them to MT4 and broker, ingests results, updates positions, classifies outcomes, and decides on reentry:

1. **Configuration snapshot (F1)** sets global parameters used everywhere.
2. **Scheduler (F3)** triggers calendar polling (D2) and anticipator (D4).
3. **Calendar ingestion (D2 → D3 → F2 → D4)** produces triggers to inform the feature frame and reentry decisions.
4. **Market feed ingestion (D1 → C1 → C2 → C3)** transforms ticks into bars, indicators and features.
5. **Signal engine (S1)** uses features and positions to create entry signals.
6. **Intent builder (S2)** makes trade intents; risk evaluator (R1) sizes them; order compiler (R2) prepares them; router (O1) selects a broker route.
7. **Transport (B1)** serializes orders to the MT4 EA; **EA executor (B2)** executes on the broker and emits events.
8. **Normalization (B3)** converts broker events into canonical reports; the **OMS (O2)** updates orders and positions; the **trade close classifier (O3)** calculates PnL and reasons.
9. **Outcome bucketer (E1)**, **proximity evaluator (E2)**, **matrix lookup (E3)** and **reentry builder (E4)** decide follow-on trades.
10. **Loop orchestrator** repeats the risk→order→execute path for reentries.
11. **Health aggregator (P1)** monitors modules and can suppress new trades if issues arise.

Areas of overlap or tight coupling

Several modules overlap or interact tightly, suggesting potential consolidation:

- **Multiple persistence layers:** Calendar events use `F2_EVENT_LOG`, while OMS, PnL and matrix data may use separate stores. A unified event and state persistence layer could reduce complexity.
- **Signal and reentry suppression logic:** Both the signal engine and reentry builder use suppression codes; a common suppression subsystem might centralize logic and reason codes.
- **Identifier lineage:** The pipeline uses `correlation_id`, `client_order_id`, `idempotency_key` and (in the implementation) `hybrid_id` and `file_seq`. These identifiers are generated and propagated in multiple places (S2, R2, B1). A single identifier service or library to assign and attach all necessary IDs could prevent errors [\[turn3file8†L9-L21\]](#) [\[turn3file9†L74-L90\]](#).
- **Transport complexity:** The MT4 adapter transport handles file sequencing, checksums, CSV representation and socket fallbacks. The specification emphasises `file_seq`, `checksum` and heartbeat logic [\[turn3file8†L23-L44\]](#), meaning the transport module is more complex than a simple adapter. There may be overlap with the broker router (O1) and orchestrator. Collapsing these into a single `transport/gateway` service could simplify error handling.
- **Reentry decision chain:** E1–E4 and the loop orchestrator create a complex post-trade decision pipeline. In minimal deployments, reentry may not be necessary, or could be turned off. It shares logic with the primary signal engine (S1) and risk evaluation; consolidating or modularising reentry separately would reduce coupling.

Simplification opportunities

To create a **cleaner directory structure and faster path to minimum viable functionality**, consider these changes:

1. **Merge calendar ingestion into a single module.** Combine D2, D3, F2 and D4 into a `calendar_ingestor` service that polls sources, normalizes events, logs them and computes triggers. This reduces four separate modules to one cohesive ingestion pipeline.
2. **Combine indicator and feature packaging.** C1, C2 and C3 could live inside a `market_processor` service that aggregates ticks, computes indicators and emits feature frames. This unifies tick → bar → indicator → feature logic into a single component with clear sub-functions.
3. **Unify intent compilation, sizing and routing.** S2, R1, R2 and O1 represent consecutive steps that could be encapsulated in a `trade_decision` service: it takes a `Signal`, consults risk policies, sizes, assigns client/order IDs and selects a broker route. Exposing a single API simplifies orchestration and allows for easier testing.
4. **Simplify transport.** Move `B1_MT4_ADAPTER_TRANSPORT` responsibilities (sequence, checksum, CSV vs socket) into a dedicated `gateway` library and decouple from orchestrator logic. Keep the EA executor (B2) unchanged but ensure it communicates via a single, well-defined interface. If only one transport mode is needed for MVP, drop support for both file and socket, choose one and remove failover complexity [turn3file9†L74-L90] .
5. **Defer reentry and outcome modules.** The outcome bucketizer (E1), proximity evaluator (E2), matrix lookup (E3), reentry intent builder (E4) and looping logic (Step 25) are primarily future-state features for reinforcement learning or sophisticated reentry strategies [turn2file0†L97-L107] . For MVP, these can be stubbed or disabled; trades close simply end the pipeline. Keeping the modules but disabling them simplifies early deployment.
6. **Centralise health monitoring.** A single `health_monitor` module could replace P1 and integrate with orchestration for circuit-breaker functionality. Lightweight health checks can be inlined in each critical service rather than a separate aggregator.
7. **Refactor the directory structure.** Align file/folder names with modules. Currently some entrypoint files don't match module names (e.g., `EAFX.mq4` vs `HUEY_P_EA_ExecutionEngine_v7.1_Enhanced.mq4`) [turn2file0†L97-L107] . Create directories per service (`calendar_ingestor/`, `market_processor/`, `trade_decision/`, `execution_gateway/`, `oms/` etc.) with clear ownership and make missing entrypoints explicit (or mark them planned). This improves traceability and reduces confusion [turn2file0†L97-L107] .

Minimum viable trading functionality (essential vs. non-essential)

Based on the process flow and responsibilities, the **minimum viable functionality** to run a basic automated trading system includes the modules marked **essential** below. Non-essential components can be simplified or temporarily disabled for early releases.

Module	Essential?	Rationale
F1_CONFIG_PREFERENCES	☐	Required to supply a consistent configuration snapshot to all modules.
F3_CLOCK_SCHEDULER	☐	Required for polling calendars (if used) and driving time-based logic; could be minimal if the calendar ingestion is simplified.
D2-D4 (Calendar ingestion)	☐ / optional	Not strictly needed for price-only strategies; can be deferred or simplified.
D1_MARKET_FEED_ADAPTER	☐	Required to ingest live market data.
C1_BAR_BUILDER	☐	Required for candle-based strategies; could be simplified to pass ticks directly to indicators if not using bars.
C2_INDICATOR_ENGINE	☐	Core for generating signals based on indicators.
C3_FEATURE_PACKAGER	☐	Assembles indicators and triggers; can be integrated into the indicator engine for MVP.
S1_SIGNAL_ENGINE	☐	Core logic to decide whether to enter a trade.
S2_INTENT_BUILDER	☐	Converts signals to a trade intent with rationale.
R1_RISK_EVALUATOR	☐	Critical to enforce risk controls and sizing.
R2_ORDER_INTENT_COMPILER	☐	Prepares broker-ready order objects.
O1_ORDER_ROUTER	☐	Chooses the broker/exchange route; may be simplified if only one broker is supported.
B1_MT4_ADAPTER_TRANSPORT	☐	Required to send orders to the execution environment (choose one transport mechanism for MVP).
B2_MT4_EA_EXECUTOR	☐	Executes orders with the broker.
B3_EXEC_EVENT_NORMALIZER	☐	Normalizes execution results to canonical form.
O2_OMS_STATE_MACHINE	☐	Maintains order and position state.
O3_TRADE_CLOSE_CLASSIFIER	☐	Computes PnL and close reasons; may be combined with OMS for MVP.

Module	Essential?	Rationale
E1_OUTCOME_BUCKETIZER	☐	Used for RL training and reentry; not needed for MVP.
E2_PROXIMITY_EVALUATOR	☐	Only required for calendar-driven reentry logic.
E3_MATRIX_LOOKUP	☐	Future-state feature; not required for MVP.
E4_REENTRY_INTENT_BUILDER	☐	Non-essential; reentry can be turned off.
Loop orchestrator	☐ (simplified)	A basic orchestrator is required to connect modules but can omit reentry loops.
P1_HEALTH_AGGREGATOR	☐	Basic health monitoring is useful; can be simplified to simple log alerts.

In summary, a **minimum viable trading system** could comprise: configuration handling, tick ingestion, bar/indicator calculation, signal generation, risk sizing, order compilation and routing, a single MT4 transport and EA executor, normalization of execution results, and position management. Calendar ingestion, outcome bucketing, proximity, matrix lookup, reentry builder and complex health aggregation can be deferred or implemented later. Consolidating adjacent modules (e.g., market processing, trade decisioning, transport/gateway) would simplify the directory structure and accelerate delivery.